Alternativas consideradas:

- **Solo la aplicación escribe la bitácora:** simple, pero cualquier acceso directo a la base de datos (HeidiSQL, otro programa) se salta el registro.
- **Servicio de auditoría en la nube:** con costo recurrente (prohibido por RA-02).
- **Triggers en la base de datos + registro de rechazos desde el servicio.**

Decisión

Decidimos que la base de datos garantice la bitácora con triggers: toda transferencia que se inserte o cambie deja su fila automáticamente; la bitácora no se puede modificar ni borrar; y cada fila guarda una huella SHA-256 encadenada con la anterior. Los intentos rechazados se guardan así: si la base de datos o la validación impiden crear la transferencia, el servicio escribe la fila RECHAZADA en la bitácora en una transacción propia (REQUIRES_NEW); si el núcleo la rechaza (saldo insuficiente), la transferencia se guarda con estado RECHAZADA. Todo con funciones gratuitas de MariaDB.

Consecuencias / Trade-offs

- **Positivas:** ninguna transferencia queda sin evidencia, sin importar por dónde entre; los rechazados nunca se pierden; la regla no depende de que un programador "se acuerde".
- **Negativas:** parte de la lógica vive en la base de datos (triggers y un procedimiento), que es menos visible que el código Java. Cada inserción hace una escritura extra en la bitácora.
- **Deuda técnica asumida:** el procedimiento registrar_transferencia repite en SQL la regla del TransferenciaService, para poder operar y demostrar desde HeidiSQL. Condición para revisarla: si las dos versiones empiezan a diferir, se deja una sola y la otra la llama.

ADR-004: En el CRUD de Transferencia, actualizar (PUT) es "corregir y reintentar"

Estatus

Aceptada

Fecha

2026-11-06

Contexto

La plantilla exige un CRUD completo de la entidad principal de la HU-01 (Transferencia). Pero una transferencia no es un dato cualquiera: si ya movió dinero en el núcleo, cambiar su monto o su destino en SITRA dejaría la evidencia diciendo algo distinto de lo que pasó. Además, en la Clase 5 se identificó un hueco de fiabilidad: si el núcleo no respondía (FALLIDA, 503), la billetera no podía reintentar con el mismo código de operación, porque el código ya existía (409).

Atributos en juego: **AC-01 Seguridad** (la evidencia debe coincidir con lo ocurrido) y **AC-02 Fiabilidad** (poder recuperarse de un fallo del núcleo sin duplicar operaciones).

Alternativas consideradas:

- **PUT libre sobre cualquier campo:** cumple la plantilla, pero permite "maquillar" una transferencia confirmada.
- **No tener PUT:** incumple el CRUD completo.
- **PUT solo sobre lo que no movió dinero (RECHAZADA o FALLIDA):** corregir destino o monto y volver a enviarla al núcleo con el mismo código de operación.

Decisión

Decidimos que PUT /api/v1/transferencias/{id} solo acepte transferencias RECHAZADAS o FALLIDAS: corrige cuentaDestino y monto, conserva el código de operación y vuelve a pasar por el núcleo (200 si queda CONFIRMADA, 422 si el núcleo la vuelve a rechazar, 503 si sigue sin responder). Sobre una CONFIRMADA responde 409. Y DELETE es un borrado lógico (ANULADA, activo = false, 204): si la transferencia ya había movido dinero, primero se pide el reverso al núcleo; si el núcleo no responde, no se anula nada (503).

Consecuencias / Trade-offs

- **Positivas:** el UPDATE tiene sentido de negocio; resuelve el reintento después de un 503 sin romper la idempotencia; la bitácora (trigger de UPDATE) guarda el intento original y el reintento, así que nada se borra ni se maquilla.
- **Negativas:** el PUT no es "genérico" como en el ejemplo de la plantilla; quien lo use debe conocer la regla (está en el README y la colección Postman la prueba: 409 y 200).
- **Deuda técnica asumida:** el reverso depende de que el núcleo real ofrezca una operación para devolver dinero; hoy solo existe en el simulador. Condición para revisarla: al implementar NucleoLegadoConector (ADR-001), confirmar con el cliente cómo se reversa una operación.

ADR-005: Token JWT firmado con HMAC del JDK y secretos con BCrypt, sin Spring Security completo

Estatus

Aceptada — reemplaza la identificación provisional por encabezado X-Cliente-Id del arranque (Clase 5).

Fecha

2026-11-06

Contexto

El consumidor de SITRA es otro sistema (la billetera), no una persona con navegador. Cada petición debe probar quién la hace, porque ese dato queda en la bitácora como responsable (**AC-01**: responsabilidad y no repudio; meta: 401 en el 100 % de los intentos sin token). En el arranque se usó el encabezado X-Cliente-Id, que cualquiera puede falsificar.

Restricciones: cero presupuesto (RA-02) y cero personal técnico (RA-03): no hay quién instale ni opere un servidor de identidad.

Alternativas consideradas:

- **Sesiones en servidor:** pensadas para navegadores; no aplican a un sistema que llama a otro.
- **Servidor de identidad (por ejemplo Keycloak, OAuth2):** muy completo, pero es otra plataforma que instalar, actualizar y operar.
- **Spring Security completo + librería JWT externa:** estándar, pero agrega una cadena de filtros y configuración que un mantenedor sin experiencia difícilmente entiende.
- **Token JWT HS256 firmado con javax.crypto (incluido en Java) + BCrypt de spring-security-crypto + un filtro propio.**

Decisión

Decidimos usar JWT HS256 firmado con HMAC-SHA256 del propio JDK (JwtServicio), validado en un solo filtro (FiltroJwt) antes de llegar a los controladores, y guardar los secretos de los clientes solo como hash BCrypt (spring-security-crypto, sin el resto de Spring Security), porque AC-01 exige identificar a quien llama en cada petición y RA-02/RA-03 exigen no agregar plataformas ni productos. El secreto de firma llega por la variable SITRA_JWT_SECRET (mínimo 32 caracteres); si no existe, se genera uno aleatorio al arrancar. El token vence a los 30 minutos.

Consecuencias / Trade-offs

- **Positivas:** 401 sin token en todas las rutas /api/** (salvo /api/v1/auth/token); el rol viaja firmado y no se puede cambiar sin invalidar la firma (probado en JwtServicioTest); los secretos nunca se guardan en texto plano; no hay nada nuevo que instalar.
- **Negativas:** código de seguridad propio (unas 240 líneas en el paquete security) que hay que cuidar: comparación de firmas en tiempo constante, se rechaza cualquier algoritmo distinto de HS256 y se valida el vencimiento. No hay renovación de tokens (refresh) ni lista de revocación; se mitiga porque en cada petición se verifica que el cliente siga activo en la base de datos: desactivarlo bloquea sus tokens de inmediato.
- **Deuda técnica asumida:** HTTPS. En el laboratorio la API corre en HTTP (puerto 8080) y el token viaja sin cifrar. Condición para revisarla: antes de producción se activa HTTPS en Spring Boot con un certificado gratuito y se rota SITRA_JWT_SECRET.

SECCIÓN 5: ENTREGABLE DE PROGRAMACIÓN EJECUTABLE (CORTE VERTICAL)

El corte vertical implementa el CRUD completo de Transferencia, la entidad principal de la HU-01, atravesando todas las capas: API REST → filtro JWT → controlador → servicio → repositorio/ORM → MariaDB (con triggers) → y de regreso. Además expone la consulta de saldo (HU-02) y la consulta y verificación de la bitácora (HU-03).

5.1 Repositorio Git

```
sitra/
├── docs/            DAS.docx, DAS.pdf, adr/ (ADR-001 a ADR-005), diagramas/ (.puml + .png)
├── db/
│   ├── migrations/  V1 tablas · V2 triggers de bitácora · V3 rechazados · V4 IP de origen
│   ├── datos_prueba.sql   clientes (secretos BCrypt), asociados, cuentas y saldos
│   ├── instalar_todo.sql  todo lo anterior en un solo archivo (F9 en HeidiSQL)
│   ├── pruebas/demo_sustentacion.sql   demostración paso a paso en HeidiSQL
│   └── produccion/usuario_minimo_privilegio.sql
├── src/main/java/co/coop/sitra/
│   ├── security/     FiltroJwt, JwtServicio, ConfiguracionSeguridad
│   ├── controller/   Auth, Transferencia, Saldo y Auditoria Controller
│   ├── service/      Transferencia, Cuenta, Auditoria e Identificacion Service
│   ├── repository/   repositorios Spring Data JPA
│   ├── model/        entidades JPA (= tablas) y enumeraciones
│   ├── integration/  NucleoConector + simulador + conector legado
│   ├── dto/          cuerpos JSON de entrada y salida
│   └── exception/    ManejadorErrores: errores → códigos HTTP
├── src/main/resources/application.properties   lee las variables de entorno
├── src/test/java/co/coop/sitra/   pruebas unitarias e integración (5.4)
├── postman/coleccion-corte-vertical.json        19 peticiones con tests
├── evidencias/      una carpeta por clase
├── .env.example  ·  .gitignore  ·  pom.xml  ·  README.md
```

Política de commits: commits atómicos con formato Conventional Commits (feat:, fix:, docs:, test:, chore:). El trabajo de cada clase se hace en una rama feature/* que se une a main al terminar, así main siempre queda ejecutable.
El .gitignore excluye .env, target/ y los archivos del editor: el archivo .env con valores reales nunca entra al historial.

5.2 Script SQL DDL / Migraciones (MariaDB)

Cuatro scripts en orden. V1 crea las tablas; V2 los triggers de la bitácora; V3 el procedimiento que registra también los rechazados; V4 es el ALTER que agrega la IP de origen a la transacción (cambio pedido en la Clase 4).
db/instalar_todo.sql los ejecuta todos de una vez junto con los datos de prueba. A continuación, V1 completo:

```
-- ================================================================
-- SITRA — V1: tablas, llaves, CHECK, UNIQUE e índices
-- Motor: MariaDB 10.4+ (XAMPP). Ejecutar en orden: V1, V2, V3, V4.
-- ================================================================

-- Quién usa SITRA: la billetera externa y el auditor
CREATE TABLE cliente_api (
    id           BIGINT        NOT NULL AUTO_INCREMENT PRIMARY KEY,
    nombre       VARCHAR(120)  NOT NULL,
    client_id    VARCHAR(80)   NOT NULL UNIQUE,
    secret_hash  VARCHAR(255)  NOT NULL,                          -- BCrypt, nunca el secreto
    rol          VARCHAR(20)   NOT NULL DEFAULT 'BILLETERA',
    activo       BOOLEAN       NOT NULL DEFAULT TRUE,             -- borrado lógico
    created_at   DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at   DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT ck_cliente_rol CHECK (rol IN ('BILLETERA', 'AUDITOR'))
```

```
) ENGINE = InnoDB;

-- Asociado de la cooperativa
CREATE TABLE asociado (
    id          BIGINT        NOT NULL AUTO_INCREMENT PRIMARY KEY,
    documento   VARCHAR(30)   NOT NULL UNIQUE,
    nombre      VARCHAR(120)  NOT NULL,
    activo      BOOLEAN       NOT NULL DEFAULT TRUE,                  -- borrado lógico
    created_at  DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at  DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE = InnoDB;

-- Cuenta (lista blanca de cuentas expuestas). NO guarda saldo: el saldo vive en el núcleo.
CREATE TABLE cuenta_autorizada (
    id             BIGINT       NOT NULL AUTO_INCREMENT PRIMARY KEY,
    asociado_id    BIGINT       NOT NULL,
    numero_cuenta  VARCHAR(30)  NOT NULL UNIQUE,
    tipo           VARCHAR(20)  NOT NULL,
    activo         BOOLEAN      NOT NULL DEFAULT TRUE,                -- borrado lógico
    created_at     DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at     DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_cuenta_asociado FOREIGN KEY (asociado_id) REFERENCES asociado(id),
    CONSTRAINT ck_cuenta_tipo CHECK (tipo IN ('AHORROS', 'CORRIENTE'))
) ENGINE = InnoDB;

-- Transacción (transferencia)
CREATE TABLE transferencia (
    id                  BIGINT         NOT NULL AUTO_INCREMENT PRIMARY KEY,
    cliente_api_id      BIGINT         NOT NULL,
    cuenta_origen_id    BIGINT         NOT NULL,
    cuenta_destino      VARCHAR(30)    NOT NULL,
    monto               DECIMAL(14,2)  NOT NULL,
    estado              VARCHAR(20)    NOT NULL DEFAULT 'PENDIENTE',
    referencia_externa  VARCHAR(80)    NOT NULL,                       -- código de operación
    comprobante_nucleo  VARCHAR(80)    NULL,
    motivo_rechazo      VARCHAR(255)   NULL,                           -- por qué se rechazó
    activo              BOOLEAN        NOT NULL DEFAULT TRUE,          -- borrado lógico
    created_at          DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at          DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_transf_cliente FOREIGN KEY (cliente_api_id)   REFERENCES cliente_api(id),
    CONSTRAINT fk_transf_cuenta  FOREIGN KEY (cuenta_origen_id) REFERENCES cuenta_autorizada(id),
    CONSTRAINT uq_transf_idempotencia UNIQUE (referencia_externa),    -- llave de idempotencia
    CONSTRAINT ck_transf_monto  CHECK (monto > 0),                    -- CHECK de monto > 0
    CONSTRAINT ck_transf_estado CHECK (estado IN ('PENDIENTE','CONFIRMADA','RECHAZADA','FALLIDA','ANULADA'))
) ENGINE = InnoDB;

-- Bitácora de auditoría (obligatoria: la trazabilidad es el atributo crítico).
-- Guarda también los intentos RECHAZADOS: código, monto intentado y motivo.
-- monto NO tiene CHECK: debe poder guardar lo que se intentó, aunque sea inválido (ej. -50.000).
CREATE TABLE registro_auditoria (
    id                  BIGINT         NOT NULL AUTO_INCREMENT PRIMARY KEY,
    cliente_api_id      BIGINT         NOT NULL,                      -- quién
    transferencia_id    BIGINT         NULL,                          -- vacío si la transferencia no se pudo crear
    numero_cuenta       VARCHAR(30)    NOT NULL,                      -- sobre qué cuenta
    operacion           VARCHAR(30)    NOT NULL,                      -- qué
    resultado           VARCHAR(20)    NOT NULL,                      -- EXITOSA / RECHAZADA / FALLIDA
    referencia_externa  VARCHAR(80)    NULL,                          -- código de operación intentado
```

```
    monto              DECIMAL(14,2)  NULL,                          -- monto intentado
    motivo             VARCHAR(255)   NULL,                          -- por qué se rechazó
    ip_origen          VARCHAR(45)    NOT NULL DEFAULT 'local',
    ocurrido_en        DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP,  -- cuándo
    hash_anterior      CHAR(64)       NULL,
    hash_actual        CHAR(64)       NULL UNIQUE,
    CONSTRAINT fk_audit_cliente FOREIGN KEY (cliente_api_id)   REFERENCES cliente_api(id),
    CONSTRAINT fk_audit_transf  FOREIGN KEY (transferencia_id) REFERENCES transferencia(id),
    CONSTRAINT ck_audit_operacion CHECK (operacion IN
('CONSULTA_SALDO','TRANSFERENCIA','ANULACION','CONSULTA_AUDITORIA')),
    CONSTRAINT ck_audit_resultado CHECK (resultado IN ('EXITOSA','RECHAZADA','FALLIDA'))
) ENGINE = InnoDB;

-- Simulador del núcleo legado: aquí vive el SALDO (SITRA no guarda saldos)
CREATE TABLE nucleo_simulado_cuenta (
    numero_cuenta  VARCHAR(30)    NOT NULL PRIMARY KEY,
    saldo          DECIMAL(14,2)  NOT NULL,
    CONSTRAINT ck_nucleo_saldo CHECK (saldo >= 0)
) ENGINE = InnoDB;

-- Índices justificados
CREATE INDEX idx_transferencia_cuenta   ON transferencia (cuenta_origen_id);
CREATE INDEX idx_auditoria_cuenta_fecha ON registro_auditoria (numero_cuenta, ocurrido_en);
```

db/migrations/V4__ip_origen_en_transferencia.sql (extracto)

```
ALTER TABLE transferencia
    ADD COLUMN ip_origen VARCHAR(45) NULL AFTER referencia_externa;
```

Cumplimiento de los requisitos evaluables

Requisito de la plantilla	Dónde se cumple
PK en toda tabla	id BIGINT AUTO_INCREMENT PRIMARY KEY en 5 tablas; nucleo_simulado_cuenta usa numero_cuenta.
FK con REFERENCES	cuenta_autorizada → asociado; transferencia → cliente_api y cuenta_autorizada; registro_auditoria → cliente_api y transferencia.
Al menos una CHECK	monto > 0 (ck_transf_monto), estado, rol, tipo de cuenta, operacion, resultado y saldo ≥ 0.
Al menos una UNIQUE	referencia_externa = llave de idempotencia (uq_transf_idempotencia); client_id, documento, numero_cuenta, hash_actual.
Borrado lógico	Campo activo en asociado, cliente_api, cuenta_autorizada y transferencia (DELETE de la API = ANULADA + activo = false). La bitácora no lo tiene a propósito: es evidencia.
Índices justificados	idx_transferencia_cuenta (historial por cuenta) e idx_auditoria_cuenta_fecha (HU-03: el auditor consulta por cuenta y fechas, meta ≤ 5 s).

Verificación ejecutada en MariaDB (db/instalar_todo.sql + db/pruebas/demo_sustentacion.sql)

```
Paso 1  transferencia 300.000 por SQL -> bitácora: TRANSFERENCIA · EXITOSA (la escribió el trigger)
Paso 2  mismo código                  -> ERROR 1062 Duplicate entry ... 'uq_transf_idempotencia'
        monto -50.000                 -> ERROR 4025 CONSTRAINT 'ck_transf_monto' failed
Paso 4  CALL con código repetido      -> RECHAZADA · queda en la bitácora con su motivo
        CALL con -50.000              -> RECHAZADA · queda en la bitácora con su motivo
        CALL con 5.000.000            -> RECHAZADA · transacción con estado RECHAZADA + bitácora
```

```
Paso 5  verificación de la cadena    -> cadena OK · huella OK en todas las filas
        UPDATE / DELETE en bitácora  -> ERROR 1644: no se puede modificar / borrar
Usuario sitra_app (mínimo privilegio): DROP TRIGGER -> ERROR 1142 · DELETE -> ERROR 1142
Alteración simulada (sin triggers)   -> fila modificada: ALTERADO · fila borrada: la siguiente ROTA
```

5.3 Código backend por capas (Spring Boot + JPA)

Lo evaluable es la separación de capas. Cada petición entra por el filtro JWT; el controlador solo traduce a HTTP; el servicio aplica las reglas y es el único que conoce el conector; el repositorio habla con MariaDB.

Endpoints

Método y ruta	Qué hace	Respuestas
POST /api/v1/auth/token	Cambia clientId + secreto (comparado con BCrypt) por un token JWT de 30 min	200 · 401
POST /api/v1/transferencias	CREATE (HU-01): valida, pasa por el núcleo y guarda con su estado final	201 · 400 · 401 · 403 · 404 · 409 · 422 · 503
GET /api/v1/transferencias	READ: lista las transferencias activas del cliente	200 · 401 · 403
GET /api/v1/transferencias/{id}	READ: consulta una transferencia activa propia	200 · 401 · 404
PUT /api/v1/transferencias/{id}	UPDATE (ADR-004): corrige y reintenta una RECHAZADA o FALLIDA	200 · 400 · 401 · 404 · 409 · 422 · 503
DELETE /api/v1/transferencias/{id}	DELETE lógico: reverso en el núcleo si movió dinero; ANULADA + activo = false	204 · 401 · 404 · 409 · 503
GET /api/v1/cuentas/{numero}/saldo	HU-02: saldo leído en vivo del núcleo (queda CONSULTA_SALDO en la bitácora)	200 · 401 · 403 · 404 · 503
GET /api/v1/auditoria?cuenta=&desde=&hasta=	HU-03: bitácora por cuenta y rango de fechas (solo AUDITOR)	200 · 400 · 401 · 403
GET /api/v1/auditoria/verificacion	HU-03: recalcula la cadena de huellas; integra = true si nadie alteró nada	200 · 401 · 403

Controlador — src/main/java/co/coop/sitra/controller/TransferenciaController.java (extracto)

```
/** CREATE: 201 CONFIRMADA · 422 RECHAZADA por el núcleo · 503 FALLIDA (núcleo sin respuesta). */
@PostMapping
public ResponseEntity<TransferenciaResponse> registrar(@RequestAttribute(FiltroJwt.ATRIBUTO_CLIENTE) String clientId,
                                                       @RequestBody CrearTransferenciaRequest datos,
                                                       HttpServletRequest peticion) {
    Transferencia t = servicio.registrar(clientId, datos, peticion.getRemoteAddr());
    return ResponseEntity.status(codigoSegunEstado(t, HttpStatus.CREATED)).body(TransferenciaResponse.de(t));
}

/** READ: solo las activas del cliente que pregunta. */
@GetMapping
public List<TransferenciaResponse> listar(@RequestAttribute(FiltroJwt.ATRIBUTO_CLIENTE) String clientId) {
    return servicio.listar(clientId).stream().map(TransferenciaResponse::de).toList();
}

@GetMapping("/{id}")
public TransferenciaResponse obtener(@RequestAttribute(FiltroJwt.ATRIBUTO_CLIENTE) String clientId,
                                     @PathVariable Long id) {
    return TransferenciaResponse.de(servicio.obtener(clientId, id));
```

```
}

/** UPDATE: corregir y reintentar una RECHAZADA o FALLIDA (ADR-004). 200 · 422 · 503 · 409. */
@PutMapping("/{id}")
public ResponseEntity<TransferenciaResponse> actualizar(@RequestAttribute(FiltroJwt.ATRIBUTO_CLIENTE) String clientId,
                                                        @PathVariable Long id,
                                                        @RequestBody ActualizarTransferenciaRequest datos,
                                                        HttpServletRequest peticion) {
    Transferencia t = servicio.actualizar(clientId, id, datos, peticion.getRemoteAddr());
    return ResponseEntity.status(codigoSegunEstado(t, HttpStatus.OK)).body(TransferenciaResponse.de(t));
}

/** DELETE lógico: 204 y la fila queda con activo = false y estado ANULADA. */
@DeleteMapping("/{id}")
public ResponseEntity<Void> anular(@RequestAttribute(FiltroJwt.ATRIBUTO_CLIENTE) String clientId,
                                   @PathVariable Long id, HttpServletRequest peticion) {
    servicio.anular(clientId, id, peticion.getRemoteAddr());
    return ResponseEntity.noContent().build();
}
```

Servicio — src/main/java/co/coop/sitra/service/TransferenciaService.java (CREATE, extracto)

```
@Transactional
public Transferencia registrar(String clientId, CrearTransferenciaRequest datos, String ip) {
    ClienteApi cliente = identificacion.exigirRol(clientId, Rol.BILLETERA);
    String cuenta = datos.cuentaOrigen();
    if (vacio(cuenta) || vacio(datos.cuentaDestino()) || vacio(datos.referenciaExterna()) || datos.monto() == null) {
        throw rechazo(cliente, null, cuenta, Operacion.TRANSFERENCIA, datos.referenciaExterna(), datos.monto(),
                HttpStatus.BAD_REQUEST, "Faltan datos: cuentaOrigen, cuentaDestino, monto y referenciaExterna son obligatorios",
ip);
    }
    if (datos.monto().compareTo(BigDecimal.ZERO) <= 0) {
        throw rechazo(cliente, null, cuenta, Operacion.TRANSFERENCIA, datos.referenciaExterna(), datos.monto(),
                HttpStatus.BAD_REQUEST, "El monto debe ser mayor que cero", ip);
    }
    if (transferencias.existsByReferenciaExterna(datos.referenciaExterna())) {
        throw rechazo(cliente, null, cuenta, Operacion.TRANSFERENCIA, datos.referenciaExterna(), datos.monto(),
                HttpStatus.CONFLICT, "El código de operación ya existe: la transferencia no se repite", ip);
    }
    CuentaAutorizada origen = cuentas.findByNumeroCuentaAndActivoTrue(cuenta).orElse(null);
    if (origen == null) {
        throw rechazo(cliente, null, cuenta, Operacion.TRANSFERENCIA, datos.referenciaExterna(), datos.monto(),
                HttpStatus.NOT_FOUND, "La cuenta no está en la lista de cuentas autorizadas", ip);
    }

    Transferencia t = Transferencia.nueva(cliente, origen, datos.cuentaDestino(), datos.monto(),
            datos.referenciaExterna(), ip);
    ejecutarEnNucleo(t);
    try {
        return transferencias.save(t);   // INSERT: el trigger escribe la bitácora
    } catch (DataIntegrityViolationException e) {
        // Dos peticiones iguales al mismo tiempo: la llave UNIQUE de la base de datos frena la segunda
        throw rechazo(cliente, null, cuenta, Operacion.TRANSFERENCIA, datos.referenciaExterna(), datos.monto(),
                HttpStatus.CONFLICT, "La base de datos rechazó la operación (código de operación repetido)", ip);
    }
}
```